



**양준영**

Backend Engineer.

반복적인 프로세스 개선과 정교한 기능 설계에 대한 관심이 많습니다.

AI Agent를 활용하며 자동화된 워크플로우를 구축하고 기획부터 개발까지 진행하며 시간과 비용을 절약해 본 경험이 있습니다. 결제 기능의 결함을 찾아내고 기능의 안정성을 개선해본 경험이 있습니다. AI를 활용한 빠른 생산성으로 일을 효율적으로 처리하며 비즈니스 이해를 바탕으로 기능 안정성까지 고민하는 개발자가 되기 위해 노력하고 있습니다.

## About

---



1998. 07. 22



sgo722@naver.com



<https://github.com/sgo722>

## Study

---

✦ 객체지향 미션 진행 2025.03 ~ 2025.05 [github.com/OOPMISSION](https://github.com/OOPMISSION)

✦ Real My SQL 1, 2권 2025.01 ~ 2025.03 [notion.com/RealMySQL](https://notion.com/RealMySQL)

## Experience

---

🎓 세종대학교 컴퓨터공학과 2022.03 ~ 2024.02

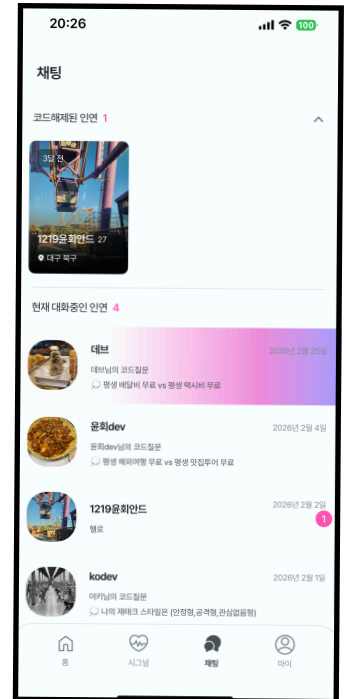
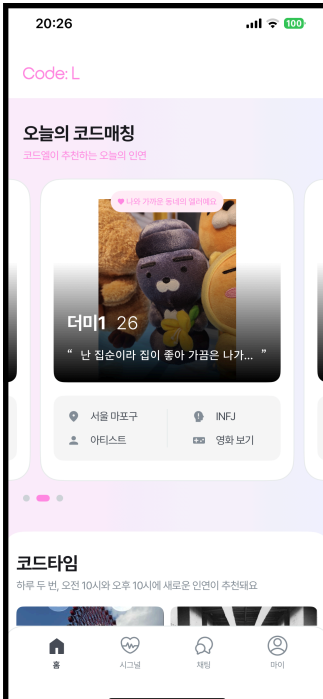
💻 아롬 세종대학교 앱개발 동아리 2023.03 ~ 2024.02

🏆 삼성청년SW 아카데미 11기 2024.01 ~ 2024.12



Code:L

2025. 07 ~ 2026.03



## 개요

'코드엘'은 가치관 기반 소셜 매칭 서비스로, 프로필 기반 매칭, 실시간 채팅, 추천 시스템을 제공합니다. 현재 200명 이상의 실사용자가 이용 중인 실서비스입니다.

## 역할

- ◆ WebSocket 기반 채팅 기능 구현
- ◆ 단일서버에서 다중서버로 전환
- ◆ AI 도구(Claude Code) 활용한 개발 자동화
- ◆ Thymeleaf 기반 관리자 페이지 개발
- ◆ 전략 패턴 기반 하위호환성 API 설계
- ◆ k6를 활용한 부하테스트

## Skills

- ◆ Kotlin, SpringBoot
- ◆ MySQL, Redis. Redis pub/sub
- ◆ AWS EC2, Grafana, Prometheus, k6

## AI 도구활용한 개발 자동화

### 개요

- CODE-L 프로젝트를 혼자 개발하면서 컨텍스트 스위칭 비용과 반복 작업이 생산성을 크게 저하시키는 문제를 발견했습니다. 이를 해결하기 위해 AI Agent를 중심으로 개발 환경과 워크플로우를 재설계하여, 개발 시간을 단축하고 코드 품질을 일관되게 유지했습니다.

### 핵심 문제

1. 컨텍스트 스위칭: 긴급 버그 수정 시 현재 작업 중단 → git stash → 복귀 후 맥락 손실 (5-10분)
2. 반복 작업: Git Issue, commit, PR 생성, 마이그레이션 검증 등에 하루 1-2시간 소모
3. AI 협업 비효율: 막연한 요청으로 원하는 결과를 얻지 못함 (정확도 60%)

### 해결 방안

- 1. 병렬 개발 환경 구축
  - tmux + git worktree로 물리적으로 분리된 작업 공간 구성
  - 기능 개발/버그 수정/실험을 독립된 세션에서 동시 진행
- 2. AI 자동화 파이프라인
  - 프롬프트 엔지니어링 (폐목형제 프레임워크)으로 AI 응답 정확도 향상
  - Claude Code 권한 설정으로 코드 생성부터 깃허브 플로우 자동화
  - 워크플로우 템플릿화 (/pr-create, /test-fix 등 커스텀 명령어)
- 3. 품질 보장 자동화
  - GitHub Actions로 모든 PR에 대해 자동 빌드/테스트 실행
  - Playwright E2E 테스트로 Thymeleaf 관리자 페이지 UI 검증 자동화

### 주요 성과

- AI 응답 정확도 향상
- 컨텍스트 스위칭 시간: 5-10분 → 0초 (100% 개선)
- 깃허브 이슈/PR 생성 시간: 10분 → 2분 (80% 단축)
- 화면 검증 시간: 30분 → 5분 (83% 단축)

### 노션 링크

- [\[AI Agent 활용 상세 내용\]](#)

## 단일 서버에서 다중 서버로의 전환

### 문제 상황

- 레즈비언 가치관 기반 소셜 매칭 서비스로 WebSocket 실시간 채팅이 핵심 기능인 서비스로 t2.micro로 시작하여 사전예약자 증가로 t3.small로 스케일업했으나, 회원 200명 도달 시점에 확장 가능한 구조의 필요성 인식
- 단일 서버는 인스턴스 장애 시 전체 중단, 장애 격리 불가, 수평 확장 불가능한 근본적 한계 존재

### 해결 방안

- SimpleBroker의 인메모리 방식으로 다중 서버 간 WebSocket 메시지 미전달
- synchronized가 JVM 독립적으로 동작하여 다중 서버 간 동시성 제어 불가
- 스케줄러가 각 서버에서 독립 실행되어 중복 알림 발송

### 문제 해결

- Redis Pub/Sub + SimpleBroker로 서버 간 WebSocket 메시지 중계
- Redisson 분산 락으로 synchronized 교체
- ShedLock으로 스케줄러 중복 실행 방지
- 로컬 전송 우선 구조로 Redis 장애 시에도 같은 서버 사용자는 정상 동작

### 성과

- 단일 장애점 제거로 인스턴스 장애 시에도 서비스 지속 가능
- 수평 확장 구조로 회원 400명 → 1,000명+ 대응 가능
- Redis 통합으로 추가 인프라 비용 0원
- 중복 추천/알림 0건, 146개 테스트 통과로 기능 무손실 검증

### 노션 링크

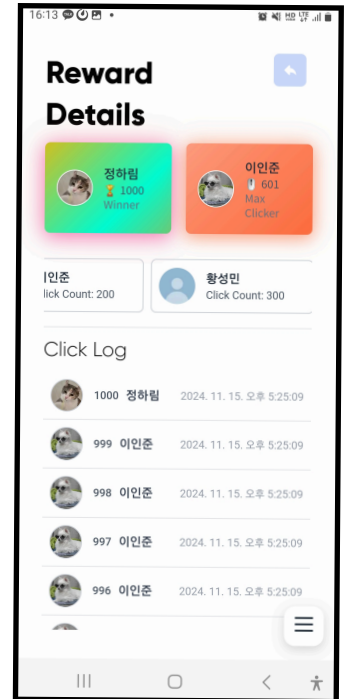
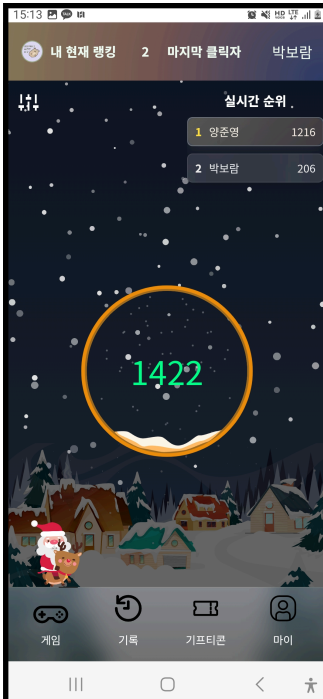
- [\[단일 서버에서 다중 서버로의 전환\]](#)





# Toudeuk

2024. 11 ~ 2024.12



## 개요 📄

'터득'은 터치해서 득템한다는 의미로, 조건에 따라 선정된 회원에게 보상을 지급하는 클릭 게임입니다. 터치 게임 기반으로 사용자에게 재미와 보상을 제공하는 서비스입니다.

## 역할 🧑

- ◆ 카카오페이 외부 API 트랜잭션 분리
- ◆ 클릭 기능 동시성 문제 해결
- ◆ 아키텍처 확장 통한 성능 개선
- ◆ 슬랙, 디스코드 알림 기능 구현
- ◆ 각 아키텍처 버전 부하테스트 비교
- ◆ 모니터링 환경 구축 ( Grafana, Prometheus, Node Exporter )

## Skills 🛠️

- ◆ Java, SpringBoot
- ◆ Redis, Kafka
- ◆ AWS EC2, Grafana, Prometheus, nGrinder

## 결제 시스템 안정성 개선

### 문제 상황

- 결제 승인 API와 아이템 지급 로직이 하나의 트랜잭션으로 묶여있어 데이터 정합성 이슈 및 외부 API 지연 시 DB 커넥션 장시간 점유 문제 발생

### 해결 방안

- 결제 상태와 아이템 지급 간의 데이터 정합성 문제 해결 필요
- 외부 API 의존으로 인한 DB 리소스 비효율적 사용
- 중복 결제/콜백/지급에 대한 방어 로직 부재

### 문제 해결

- 결제와 아이템 지급 트랜잭션 분리 (Propagation.REQUIRES\_NEW)
- 결제 내역 기반 재시도(즉시 3회) / 스케줄링(5분) / 알람(Slack)을 통한 정합성 보장
- DB 유니크 제약으로 멱등성 확보 및 중복 지급 방지
- 외부 API 호출 타임아웃 설정(3초)을 통한 DB 커넥션 자원 효율화
- 결제상태변경 테이블을 통해 상태 변경 이력 추적

### 성과

- 데이터 정합성 100% 보장
- 복구 시간 5분 → 3-4초 단축
- 중복 지급 0건

### 노션 링크

- [\[결제 시스템 안정성 개선\]](#)

## 성능 개선 경험

### 문제 상황

- 리크 게임에서 동시다발적인 클릭 요청 처리 시 심각한 성능 저하가 발생했습니다 (TPS 78).

### 해결 방안

- 비관적 락으로 인한 락 경합 및 DB 커넥션 병목
- 클릭마다 DB 직접 저장으로 응답 지연
- 쓰레드 풀/커넥션 풀 부족으로 커넥션 생성 비용 증가

### 문제 해결

- Redis INCR 도입 (TPS 78 → 328, 4.2배)
  - 비관적 락 → 락 없는 원자 연산으로 동시성 처리
  - 메모리 기반 빠른 응답
- Kafka 비동기 처리 (TPS 328 → 472, 1.4배)
  - 동기 DB 저장 → 비동기 Kafka 전송
  - DB 쓰기 부하 분산
- 쓰레드 풀/커넥션 풀 최적화 (TPS 472 → 521, 1.1배)
  - 모니터링 기반 풀 사이즈 조정

### 성과

- TPS 78 → 521 (6.5배 향상)

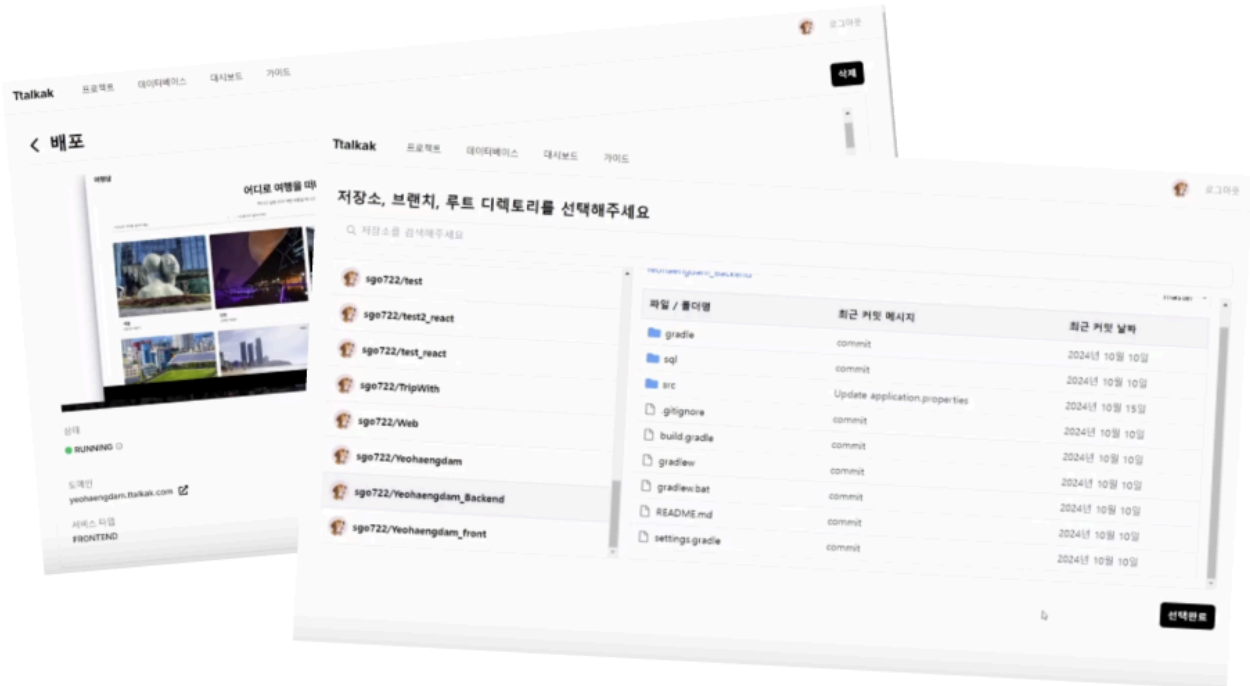
### 노션 링크

- [\[성능 개선 경험\]](#)



딸깍

2024. 09 ~ 2024. 10



## 개요

수많은 프로젝트들이 복잡한 배포 과정으로 인해 빛을 보지 못하는 현실을 목격하고 시작하게 되었습니다. 노트북 하나만 있으면 누구나 몇 번의 클릭으로 자신의 프로젝트를 손쉽게 배포할 수 있는 서비스입니다.

## 역할

- ◆ 도커파일 제작 기능 구현
- ◆ SAGA 코레오그래피 패턴 적용
- ◆ MSA 환경에서 Deployment 도메인 개발

## Skills

- ◆ Java, SpringBoot
- ◆ MySQL, Redis, Kafka
- ◆ AWS EC2, GCP

## 도커파일 자동 생성 시스템

### 문제 상황

- 프로젝트 배포를 위해 도커파일 자동 생성 기능을 개발했으나, AI가 전체 파일을 분석하여 생성하는 방식은 의존성과 빌드 과정을 정확히 파악하지 못해 성공률이 30%에 불과

### 해결 방안

- AI 단독 분석의 한계: 프로젝트 구조를 완전히 이해하지 못해 오류 발생
- 토큰 소모 비효율: 전체 파일 분석으로 인한 과도한 토큰 사용
- 일관성 부족: 같은 프로젝트 타입도 매번 다른 결과 생성

### 문제 해결

- 1차 시도(AI 전체 분석): 성공률 30-40%로 실패
- 2차 시도(프로젝트 정보 제공): AI에게 패키지 도구/빌드 도구/언어 정보 제공, 성공률 50-60%로 개선
- 3차 시도(템플릿 기반): 프로젝트 타입별 검증된 템플릿 구축 + 사용자 입력 정보 주입 방식으로 전환
- 컨텍스트 최적화: 필요한 정보만 AI에 전달하여 토큰 소모량 절감

### 성과

- 배포 성공률 30% → 90% 개선 (3배 향상)
- 토큰 사용량 대폭 감소
- 사용자 도커 전문 지식 없이도 배포 가능

### 노션 링크

- [\[도커파일 자동 생성 시스템\]](#)

## SAGA 패턴 도입

### 문제 상황

- MSA 환경에서 Project와 Deployment는 1:N 관계. Project 삭제 시 관련 Deployment들이 삭제되지 않아 불필요한 리소스가 계속 실행되며 비용 발생

### 해결 방안

- Hard Delete의 문제: 삭제 실패 시 데이터 복구 불가능
- 데이터 정합성 부재: Project는 삭제됐으나 Deployment는 남아있는 불일치 상태
- 비용 낭비: 관리되지 않는 Deployment 리소스가 계속 실행

### 문제 해결

- Soft Delete 도입: status 플래그 기반으로 논리적 삭제, 복구 가능성 확보
- SAGA 코레오그래피 패턴: Project 삭제 시 이벤트 발행 → Deployment가 구독하여 자동 삭제
- 보상 트랜잭션: 삭제 실패 시 상태 롤백으로 정합성 보장
- 서비스 분리: 서비스 간 결합도를 낮추고 장애 전파 최소화

### 성과

- Project - Deployment 데이터 정합성 100% 보장
- 리소스 비용 절감
- 삭제 실패 시 복구 가능성 확보

### 노션 링크

- [\[SAGA 패턴을 통한 분산 트랜잭션 관리\]](#)